

TOPIC 58/60

Chain of Thought Prompting

Unlocking System 2 Reasoning.

How forcing autoregressive models to generate step-by-step intermediate rationales dramatically improves math, logic, and planning execution.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



THE PROBLEM

System 1 vs. System 2

Standard autoregressive LLMs are trained to output their next token with uniform compute. If you ask a complex mathematical question, the model has to immediately compute the final answer on its very first output token.



System 1 (Intuitive)

Instant, subconscious, and direct. Great for translating words or naming capitals. Fails catastrophically on logic puzzles.



System 2 (Deliberate)

Slow, logical, step-by-step reasoning. Necessary for multi-stage math, syntax parsing, and physical simulations.

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



The Dynamic Working Memory

Why does letting a model "show its work" actually change its accuracy mathematically?

- 1 Autoregressive Context Expansion**
Every reasoning token the model writes is appended back into its dynamic input sequence, serving as physical intermediate storage.

- 2 Self-Attention Weight Calibration**
By generating step 1 first, the self-attention weights for generating step 2 have direct, concrete context vectors to index into.

- 3 Scaling Computational Steps**
Allowing 50 tokens of rationalization scales the compute allocated to finding the answer from 1 feed-forward pass to 50 passes.



TRIGGERING REASON

Zero-Shot CoT

Discovered by Kojima et al. (2022), you don't even need to give the model examples to think step-by-step. You just have to append a single linguistic key:

Prompt Setup:

```
[User Problem]: John has 5 apples. He buys 2 more boxes, each with 6 apples. How many does he have?  
[System Directive]: "Let's think step by step."
```

This phrase activates hidden probabilistic paths associated with manuals, mathematical step listings, and structured scientific reasoning, bypassing standard direct "guessing" behavior.



ADVANCED METHOD

Few-Shot CoT

First formalized by Wei et al. (2022), you can program a highly specific logical sequence by providing exact reasoning pathways inside in-context demonstrations:

```
// In-Context Exemplar with Explicit Logic
```

```
Q: Sally visited 3 shops. At shop A, she spent $10. At shop B, she spent double. Shop C was half of A. How much total?
```

```
A: Let's calculate the cost at each shop:
```

1. Shop A cost is \$10.
 2. Shop B cost is $\$10 * 2 = \20 .
 3. Shop C cost is $\$10 / 2 = \5 .
 4. Total spend is $\$10 + \$20 + \$5 = \35 .
- ```
The answer is $35.
```

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



BLUEPRINT DESIGN

# The Rationalization Blueprint

An elite CoT structural blueprint enforces structured tags to isolate calculations from final, parsed values:



## Structural Tag isolation

Instruct the model to isolate thoughts and answers to simplify parsing:

```
[Thought Block]: Wrap your intermediate calculations in <thought> </thought> tags.
[Final Extraction]: Put the final resolved value inside <answer> </answer> tags.
[Conditioning]: Never print your final conclusion until your thought tags are closed.
```

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



ADVANCED TUNING

# Self-Consistency

How do we prevent a model from making a single arithmetic slip-up that ruins the entire downstream chain of logic? We use Self-Consistency (Wang et al., 2022):

Path 1 (Temp 0.7): "Sally spent  $10 + 20 + 4 = 34$ . Answer is 34." (Calculation slip)

Path 2 (Temp 0.7): "Sally spent  $10 + 20 + 5 = 35$ . Answer is 35."

Path 3 (Temp 0.7): " Sally spent  $10 + 20 + 5 = 35$ . Answer is 35."

🇮🇳 Majority Vote Winner: 35 (2 out of 3 paths agree)

Ashish Jangra

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



## MODERN ARCHITECTURES

# Inference-Time Compute

The evolution of Chain of Thought has progressed from basic prompts to reinforcement-learned deep reasoning models like **OpenAI o1/o3** and **DeepSeek-R1**:

### Reinforcement Loop

The model is trained via RL (Reinforcement Learning) to reward itself for long-form, accurate intermediate thinking steps, correcting mistakes mid-thought.

### Internal Hidden Thoughts

Instead of forcing you to craft CoT prompts, these models automatically spin up hidden reasoning steps behind the scenes before delivering final user tokens.

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)



RISK ASSESSMENT

# Limitations & Trade-Offs

Chain of Thought is not a silver bullet. Applying it to every task introduces severe pipeline issues:

| Vector          | Impact                    | Mitigation                                         |
|-----------------|---------------------------|----------------------------------------------------|
| Token Costs     | Outputs Inflated          | Restrict CoT to tough reasoning tasks.             |
| High Latency    | Slow TTFT / EOT           | Enable streaming; run reasoning asynchronously.    |
| Naive Overshoot | Overthinking Simple Tasks | Use standard direct zero-shot for classifications. |



# Chain of Thought

CONCEPT UNLOCKED



Forces deliberate System 2 planning over reflex System 1 guesses



Generated steps act as dynamic external working memory



"Step-by-step" zero-shot cues steer autoregressive weights



Scaled globally into modern internal-reasoning AI stacks

CONNECT WITH ME



LinkedIn



GitHub



Kaggle



Instagram

**Ashish Jangra**

[linkedin.com/in/ashish-jangra](https://www.linkedin.com/in/ashish-jangra)

